

# 3D-XR\_pbd

March 16, 2026

## 1 3D XR-PBD in Elements Scenegraph

All XR-PBD simulations run fully in 3D, integrated into the Elements ECSS scenegraph.

Based on: - *MSc Thesis* — *M. Tamiolakis (2024): XR-PBD Real-Time Physics Framework - SIGGRAPH Asia 2025: XR-PBD Cross-platform Physics - Elements framework* (ECSS — Entity Component System Scenegraph)

### 1.1 How It Works

Each simulation section: 1. **Runs the XR-PBD solver** (XRPBDSimulator) for physics 2. **Creates Elements Entities** for each particle/body 3. **Updates BasicTransform.trs** every frame from XPBD positions 4. **Renders via Elements Scene** with SDL2 + OpenGL4 + ImGui

XR-PBD Particles      BasicTransform.trs      Elements RenderMesh      GPU

XPBD Constraint Solver      Scenegraph Update

### 1.2 Scenegraph Architecture

Root (Entity)

Camera (entityCam1 → entityCam2)

Floor (terrain plane)

Sim\_Rope (Entity) ← one Entity per particle

BasicTransform (trs = XPBD position)

RenderMesh (sphere geometry)

ShaderGLDecorator

Sim\_Softbody ...

...

| Section | Simulation                      | Scenegraph Entities           |
|---------|---------------------------------|-------------------------------|
| 1       | Rope                            | 16 sphere entities            |
| 2       | Flexible Rod (bending)          | 20 capsule entities           |
| 3       | Soft Body (volume)              | 4 tet vertices + 6 edge lines |
| 4       | Cloth                           | 12×12 grid of points          |
| 5       | Rigid Body (shape matching)     | 9-particle box                |
| 6       | Piercing (insertion constraint) | membrane triangle + needle    |

```
[ ]: import sys, os, pathlib
import numpy as np
import warnings
warnings.filterwarnings('ignore')

# Resolve the Physics directory robustly
# __vsc_ipynb_file__ is set by VS Code Jupyter; fall back to CWD for plain
# Jupyter.
try:
    _nb_dir = pathlib.Path(__vsc_ipynb_file__).resolve().parent
except NameError:
    _nb_dir = pathlib.Path(os.path.abspath('')).resolve()
_phys_dir = str(_nb_dir)
if _phys_dir not in sys.path:
    sys.path.insert(0, _phys_dir)

# Safety-net: make sure the Elements src/ folder is on sys.path
# Walks up 4 parents from Physics/ to Elements/, then appends src/
_elements_src = str(pathlib.Path(_phys_dir).parents[4] / 'src')
if os.path.isdir(_elements_src) and _elements_src not in sys.path:
    sys.path.insert(0, _elements_src)

from xr_pbd_extension import (
    XRPBDSimulator, Particle,
    DistanceConstraint, VolumeConstraint,
    InsertionConstraint
)

# Elements imports
import Elements.pyECSS.math_utilities as util
from Elements.pyECSS.Entity import Entity
from Elements.pyECSS.Component import BasicTransform, Camera, RenderMesh
from Elements.pyECSS.System import TransformSystem, CameraSystem
from Elements.pyGLV.GL.Scene import Scene
from Elements.pyGLV.GUI.Viewer import RenderGLStateSystem
from Elements.pyGLV.GUI.ImguiDecorator import ImGUIecssDecorator2
from Elements.pyGLV.GL.Shader import (
    InitGLShaderSystem, Shader,
    ShaderGLDecorator, RenderGLShaderSystem
)
from Elements.pyGLV.GL.VertexArray import VertexArray
from Elements.utils.Shortcuts import displayGUI_text
from OpenGL.GL import GL_LINES, GL_POINTS

print(f" Physics dir : {_phys_dir}")
print(f" Elements src : {_elements_src}")
print(" XR-PBD + Elements imports OK")
```

```
print(f" NumPy: {np.__version__}")
```

### 1.3 Geometry Helpers

Procedural 3D geometry generators for sphere/capsule particles, edges, and grids. These feed into `RenderMesh.vertex_attributes` for Elements' GPU pipeline.

```
[ ]: def make_sphere_mesh(r=0.06, lats=6, lons=8):
    """Generate a UV sphere as (vertices_Nx4, colors_Nx4, indices)."""
    verts, cols, idxs = [], [], []
    for i in range(lats+1):
        lat = np.pi/2 - np.pi * i / lats
        for j in range(lons+1):
            lon = 2*np.pi * j / lons
            x = r * np.cos(lat) * np.cos(lon)
            y = r * np.sin(lat)
            z = r * np.cos(lat) * np.sin(lon)
            verts.append([x, y, z, 1.0])
            # Color by latitude (top=bright, bottom=dark)
            t = i / lats
            cols.append([0.2+0.8*(1-t), 0.5*t, 0.9*(1-t), 1.0])
    for i in range(lats):
        for j in range(lons):
            a = i*(lons+1)+j
            idxs += [a, a+1, a+lons+1, a+1, a+lons+2, a+lons+1]
    return (np.array(verts, np.float32),
            np.array(cols, np.float32),
            np.array(idxs, np.uint32))

def make_edge_mesh(p0, p1, color=(1,0.3,0.3,1)):
    """A single edge (line segment) between two 3D points."""
    verts = np.array([[*p0, 1.0], [*p1, 1.0]], np.float32)
    cols = np.array([color, color], np.float32)
    idxs = np.array([0, 1], np.uint32)
    return verts, cols, idxs

def make_point_mesh(color=(1,1,0,1)):
    """A single point marker at origin."""
    verts = np.array([[0,0,0,1]], np.float32)
    cols = np.array([color], np.float32)
    idxs = np.array([0], np.uint32)
    return verts, cols, idxs

# Helper: create camera + scene with terrain
def build_scene_base(title="XR-PBD 3D"):
    """Build a Scene with root, camera, terrain floor. Returns (scene, root,
    ↪cam_data)."""
```

```

scene = Scene()
root = scene.world.createEntity(Entity(name="Root"))

# Camera hierarchy
eCam1 = scene.world.createEntity(Entity(name="eCam1"))
scene.world.addEntityChild(root, eCam1)
trans1 = scene.world.addComponent(eCam1, BasicTransform(name="trans1",
↪trs=util.identity()))

eye = util.vec(4.0, 3.0, 4.0)
target = util.vec(0.0, 0.0, 0.0)
up = util.vec(0.0, 1.0, 0.0)
view = util.lookat(eye, target, up)
proj = util.perspective(50.0, 1.33, 0.01, 50.0)
m = np.linalg.inv(proj @ view)

eCam2 = scene.world.createEntity(Entity(name="eCam2"))
scene.world.addEntityChild(eCam1, eCam2)
scene.world.addComponent(eCam2, BasicTransform(name="trans2", trs=util.
↪identity()))
orthoCam = scene.world.addComponent(eCam2, Camera(m, "orthoCam", "Camera",
↪"500"))

return scene, root, orthoCam, proj

SPHERE_V, SPHERE_C, SPHERE_I = make_sphere_mesh(r=0.07)
print(f" Sphere mesh: {len(SPHERE_V)} verts, {len(SPHERE_I)//3} tris")
print(" Scene helpers ready")

```

```

[ ]: def add_particle_entity(scene, root, name, pos, sphere_v, sphere_c, sphere_i,
↪color=None):
    """
    Add a single particle entity to the Elements scenegraph at given 3D
    ↪position.
    Returns (entity, transform, shader_decorator).
    """
    ent = scene.world.createEntity(Entity(name=name))
    scene.world.addEntityChild(root, ent)
    trs = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
    trans = scene.world.addComponent(ent, BasicTransform(name=f"{name}_t",
    ↪trs=trs))
    mesh = scene.world.addComponent(ent, RenderMesh(name=f"{name}_m"))

    # Optionally re-colour sphere
    vc = sphere_c.copy()
    if color is not None:
        vc[:] = color

```

```

mesh.vertex_attributes.append(sphere_v)
mesh.vertex_attributes.append(vc)
mesh.vertex_index.append(sphere_i)
scene.world.addComponent(ent, VertexArray())
shader = scene.world.addComponent(ent,
    ShaderGLDecorator(Shader(
        vertex_source = Shader.COLOR_VERT_MVP,
        fragment_source= Shader.COLOR_FRAG)))
return ent, trans, shader

def add_edge_entity(scene, root, name, p0, p1):
    """Add a line-segment edge entity between two positions."""
    ent = scene.world.createEntity(Entity(name=name))
    scene.world.addEntityChild(root, ent)
    trans = scene.world.addComponent(ent, BasicTransform(name=f"{name}_t",
    ↪trs=util.identity()))
    mesh = scene.world.addComponent(ent, RenderMesh(name=f"{name}_m"))
    ev, ec, ei = make_edge_mesh(p0, p1)
    mesh.vertex_attributes.append(ev)
    mesh.vertex_attributes.append(ec)
    mesh.vertex_index.append(ei)
    scene.world.addComponent(ent, VertexArray(primitive=GL_LINES))
    shader = scene.world.addComponent(ent,
        ShaderGLDecorator(Shader(
            vertex_source = Shader.COLOR_VERT_MVP,
            fragment_source= Shader.COLOR_FRAG)))
    return ent, trans, shader, ev

def update_transform(trans, shader, mvp):
    """Push updated MVP to a particle's shader uniform."""
    shader.setUniformVariable(key='modelViewProj', value=mvp, mat4=True)

print(" Entity factory functions ready")

```

## 1.4 Section 1: 3D Rope Simulation in Elements Scenegraph

**XR-PBD** drives the particles; each particle maps to a **sphere Entity** in the scenegraph.

Every frame: 1. `sim.step(dt)` → updates `particle.pos` 2. `BasicTransform.trs = translate(pos.x, pos.y, pos.z)` 3. `mvp = proj @ view @ trs` → GPU uniform

The **line edges** (`VertexArray GL_LINES`) connect consecutive particles and are updated by rewriting their vertex buffers.

```

[ ]: # 3D Rope Demo
import imgui

def run_rope_3d(n_rope=16, sim_steps=500, dt=0.016):

```

```

"""3D rope simulation in Elements scenegraph. Hit ESC to exit."""
# 1. Build XR-PBD simulation
sim = XRPBDSimulator(substeps=8, iterations=6, sor_factor=1.8,
                      gravity=[0., -9.81, 0.])
ids = sim.add_rope(n=n_rope, length=3.5, origin=(0., 3.5), alpha=1e-4)
# Rope is 2D internally - expand to 3D
for i, pid in enumerate(ids):
    pt = sim.particles[pid]
    pt.pos = np.array([pt.pos[0], pt.pos[1], 0.0], dtype=float)
    pt.prev_pos = pt.pos.copy()
    pt.vel = np.zeros(3)
sim.gravity = np.array([0., -9.81, 0.])
# Swing impulse at mid-point (in 3D)
sim.particles[ids[8]].vel = np.array([4.0, 0.0, 2.0])

# 2. Build Elements Scene
scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Rope")

# Add Systems
trans_sys = scene.world.
↪createSystem(TransformSystem("ts", "TransformSystem", "001"))
cam_sys = scene.world.
↪createSystem(CameraSystem("cs", "CameraSystem", "200"))
render_sys = scene.world.createSystem(RenderGLShaderSystem())
init_sys = scene.world.createSystem(InitGLShaderSystem())

# 3. Scenegraph: one entity per particle
particle_data = [] # list of (trans, shader)
for i, pid in enumerate(ids):
    pos = sim.particles[pid].pos
    col = np.tile([0.2, 0.8, 1.0, 1.0], (len(SPHERE_V), 1)).astype(np.
↪float32)
    if sim.particles[pid].inv_mass == 0:
        col[:] = [1.0, 0.5, 0.0, 1.0] # pin = orange
    ent, tr, sh = add_particle_entity(scene, root, f"rope_{i}", pos,
                                     SPHERE_V, SPHERE_C, SPHERE_I)
    particle_data.append((tr, sh))

# 4. Edge entities (lines between consecutive particles)
edge_data = []
for i in range(n_rope - 1):
    p0 = sim.particles[ids[i]].pos
    p1 = sim.particles[ids[i+1]].pos
    ent_e, tr_e, sh_e, ev = add_edge_entity(scene, root, f"edge_{i}", p0,
↪p1)
    edge_data.append((tr_e, sh_e, ev, ent_e))

```

```

# 5. Init GL
scene.init(imgui=True, windowWidth=1024, windowHeight=768,
           windowTitle="XR-PBD 3D Rope - Elements Scenegraph",
           customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager          = scene.world.eventManager
gWindow           = scene.renderWindow
rgl_actuator      = RenderGLStateSystem()
eManager._subscribers['OnUpdateCamera'] = gWindow
eManager._actuators['OnUpdateCamera']   = rgl_actuator
eManager._subscribers['OnUpdateWireframe'] = gWindow
eManager._actuators['OnUpdateWireframe'] = rgl_actuator
gWindow._myCamera = util.lookat(util.vec(4,3,4), util.vec(0,0,0), util.
↪vec(0,1,0))

# 6. Run loop
step = 0; running = True
while running:
    running = scene.render()

    # XPBD physics step
    sim.step(dt)
    view = gWindow._myCamera

    # Update particle transforms
    for i, (tr, sh) in enumerate(particle_data):
        pos = sim.particles[ids[i]].pos
        mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
        mvp = proj @ view @ mdl
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=mvp, mat4=True)

    # Update edge lines
    for i, (tr_e, sh_e, ev, ent_e) in enumerate(edge_data):
        p0 = sim.particles[ids[i]].pos
        p1 = sim.particles[ids[i+1]].pos
        ev[0,:3] = p0; ev[1,:3] = p1
        mvp_e = proj @ view @ util.identity()
        sh_e.setUniformVariable(key='modelViewProj', value=mvp_e, mat4=True)

    # ImGui overlay
    displayGUI_text(
        f"XR-PBD 3D Rope\n"
        f"Particles: {n_rope} | Constraints: {len(sim.constraints)}\n"
        f"Step: {step} | dt={dt*1000:.0f}ms\n"
        f"ESC to exit"

```

```

    )
    scene.world.traverse_visit(render_sys, scene.world.root)
    scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
    scene.world.traverse_visit(cam_sys, scene.world.root)
    scene.render_post()
    step += 1

scene.shutdown()
print(f" 3D Rope simulation finished after {step} steps")

print("run_rope_3d() defined - runs an interactive Elements window")
print("Call: run_rope_3d()")

```

```

[ ]: # Uncomment to launch interactive 3D rope window:
run_rope_3d()

```

## 1.5 Section 2: 3D Soft Body — Tetrahedral Volume Constraint in Elements

A **tetrahedron** with 4 particle vertices, 6 edge constraints (distance), and 1 volume constraint. Each of the 4 vertices is an Entity; the 6 edges are GL\_LINES entities. Gravity pulls it down, the volume constraint prevents it from collapsing.

```

[ ]: def run_softbody_3d(sim_steps=600, dt=0.014):
    """3D tetrahedral soft body in Elements scenegraph."""
    sim = XRPBDSimulator(substeps=10, iterations=12, sor_factor=1.8,
                          gravity=[0., -9.81, 0.])

    # Tetrahedron vertices (centered, above origin)
    verts = np.array([[0,0,0],[0.9,0,0],[0.45,0,0.78],[0.45,0.735,0.26]], float)
    verts -= verts.mean(0); verts[:,1] += 2.2
    pids = [sim.add_particle(v.tolist(), 1.0) for v in verts]
    EMAP = [(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)]
    for a,b in EMAP:
        sim.add_constraint(DistanceConstraint(pids[a], pids[b], sim.particles,
        ↪alpha=1e-4))
    vc = VolumeConstraint(pids[0], pids[1], pids[2], pids[3], sim.particles,
    ↪alpha=1e-4)
    sim.add_constraint(vc)

    # Build Elements Scene
    scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Soft Body")
    trans_sys = scene.world.
    ↪createSystem(TransformSystem("ts","TransformSystem","001"))
    cam_sys = scene.world.
    ↪createSystem(CameraSystem("cs","CameraSystem","200"))
    render_sys = scene.world.createSystem(RenderGLShaderSystem())
    init_sys = scene.world.createSystem(InitGLShaderSystem())

```



```

# Vertex sphere entities
colors = [[0.9,0.2,0.2,1],[0.2,0.9,0.2,1],[0.2,0.2,0.9,1],[0.9,0.9,0.2,1]]
vert_data = []
for i, pid in enumerate(pids):
    pos = sim.particles[pid].pos
    col = np.tile(colors[i], (len(SPHERE_V),1)).astype(np.float32)
    ent, tr, sh = add_particle_entity(scene, root, f"tet_{i}", pos,
                                     SPHERE_V, SPHERE_C, SPHERE_I,
↳color=col)
    vert_data.append((tr, sh, pid))

# Edge entities (lines)
edge_data = []
for ei,(a,b) in enumerate(EMAP):
    p0 = sim.particles[pids[a]].pos
    p1 = sim.particles[pids[b]].pos
    ent_e, tr_e, sh_e, ev = add_edge_entity(scene, root, f"tet_e{ei}", p0,
↳p1)
    edge_data.append((tr_e, sh_e, ev, a, b))

scene.init(imgui=True, windowWidth=1024, windowHeight=768,
           windowTitle="XR-PBD 3D Soft Body - Volume + Distance,
↳Constraints",
           customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager = scene.world.eventManager; gWindow = scene.renderWindow
rgl = RenderGLStateSystem()
for ev_name in ['OnUpdateCamera','OnUpdateWireframe']:
    eManager._subscribers[ev_name] = gWindow
    eManager._actuators[ev_name] = rgl
gWindow._myCamera = util.lookat(util.vec(3,2.5,3),util.vec(0,1,0),util.
↳vec(0,1,0))

step = 0; running = True
while running:
    running = scene.render()
    sim.step(dt)
    view = gWindow._myCamera

# Ground bounce
for pid in pids:
    pt = sim.particles[pid]
    if pt.pos[1] < -1.5:
        pt.pos[1] = -1.5
        pt.vel[1] = abs(pt.vel[1]) * 0.45

```

```

# Volume conservation check (every 100 steps)
p = [sim.particles[pid].pos for pid in pids]
d1,d2,d3 = p[1]-p[0], p[2]-p[0], p[3]-p[0]
Vcur = abs(np.dot(d1, np.cross(d2,d3))) / 6.0
vol_err = abs(Vcur - vc.V0) / max(vc.V0, 1e-9) * 100

# Update vertex entities
for tr, sh, pid in vert_data:
    pos = sim.particles[pid].pos
    mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
    tr.trs = mdl
    sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

# Update edge entities
for tr_e, sh_e, ev, a, b in edge_data:
    ev[0,:3] = sim.particles[pids[a]].pos
    ev[1,:3] = sim.particles[pids[b]].pos
    sh_e.setUniformVariable(key='modelViewProj', value=(proj@view@util.
↪identity()), mat4=True)

displayGUI_text(
    f"XR-PBD 3D Soft Body (Tetrahedron)\n"
    f"Volume V0={vc.V0:.3f} Vcur={Vcur:.3f} err={vol_err:.1f}%\n"
    f"Step {step} | Distance+Volume Constraints | SOR w=1.8\n"
    f"ESC to exit"
)

scene.world.traverse_visit(render_sys, scene.world.root)
scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
scene.world.traverse_visit(cam_sys, scene.world.root)
scene.render_post()
step += 1

scene.shutdown()
Vf = abs(np.dot(d1, np.cross(d2,d3)))/6.0
print(f" Soft body: V0={vc.V0:.4f} Vf={Vf:.4f} err={abs(Vf-vc.V0)/vc.
↪V0*100:.2f}% ({step} steps)")

print("run_softbody_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_softbody_3d()

```

## 1.6 Section 3: 3D Cloth Simulation in Elements

**12 × 12 particle grid** with structural, shear, and bending distance constraints. Two top-corner particles are pinned. A wind impulse is applied at t=60 frames.

Each particle is rendered as a **point entity** (GL\_POINTS) and grid row/col connections as GL\_LINES. The simulation runs in a 3D XZ plane initially, but wind pushes it out-of-plane.

```
[ ]: def run_cloth_3d(rows=10, cols=10, dt=0.016):
    """3D cloth simulation in Elements scenegraph."""
    sim = XRPBDSimulator(substeps=6, iterations=8, sor_factor=1.8,
                          gravity=[0., -9.81, 0.])
    ids = sim.add_cloth_grid(rows=rows, cols=cols, width=3.0, height=3.0,
                             origin=(0.0, 3.0), mass=0.4)

    # Promote 2D particles to 3D (add z=0)
    for r in range(rows):
        for c in range(cols):
            pt = sim.particles[ids[r,c]]
            pt.pos = np.array([pt.pos[0], pt.pos[1], 0.0], dtype=float)
            pt.prev_pos = pt.pos.copy()
            pt.vel = np.zeros(3, dtype=float)
    sim.gravity = np.array([0., -9.81, 0.])

    # Build Scene
    scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Cloth")
    trans_sys = scene.world.
    ↪createSystem(TransformSystem("ts", "TransformSystem", "001"))
    cam_sys = scene.world.
    ↪createSystem(CameraSystem("cs", "CameraSystem", "200"))
    render_sys = scene.world.createSystem(RenderGLShaderSystem())
    init_sys = scene.world.createSystem(InitGLShaderSystem())

    # One small sphere entity per particle
    sV, sC, sI = make_sphere_mesh(r=0.04, lats=4, lons=6)
    part_data = []
    for r in range(rows):
        for c in range(cols):
            pid = ids[r,c]
            pos = sim.particles[pid].pos
            pinned = (sim.particles[pid].inv_mass == 0)
            col = np.tile([1.0,0.4,0.0,1.0] if pinned else [0.5,0.8,1.0,1.0],
                          (len(sV),1)).astype(np.float32)
            ent, tr, sh = add_particle_entity(scene, root, f"c_{r}_{c}", pos,
            ↪sV, sC, sI, color=col)
            part_data.append((r, c, pid, tr, sh))

    # Horizontal edge entities (row connections)
```

```

h_edges = []
for r in range(rows):
    for c in range(cols-1):
        p0 = sim.particles[ids[r,c]].pos
        p1 = sim.particles[ids[r,c+1]].pos
        _, tr_e, sh_e, ev = add_edge_entity(scene, root, f"he_{r}_{c}", p0,
↪p1)
        h_edges.append((tr_e, sh_e, ev, r, c))

# Vertical edge entities (column connections)
v_edges = []
for r in range(rows-1):
    for c in range(cols):
        p0 = sim.particles[ids[r,c]].pos
        p1 = sim.particles[ids[r+1,c]].pos
        _, tr_e, sh_e, ev = add_edge_entity(scene, root, f"ve_{r}_{c}", p0,
↪p1)
        v_edges.append((tr_e, sh_e, ev, r, c))

scene.init(imgui=True, windowWidth=1024, windowHeight=768,
            windowTitle="XR-PBD 3D Cloth - Elements Scenegraph",
            customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager = scene.world.eventManager; gWindow = scene.renderWindow
rgl = RenderGLStateSystem()
for ev_name in ['OnUpdateCamera', 'OnUpdateWireframe']:
    eManager._subscribers[ev_name] = gWindow
    eManager._actuators[ev_name] = rgl
gWindow._myCamera = util.lookat(util.vec(5,2,2),util.vec(0,0,0),util.
↪vec(0,1,0))

step = 0; running = True
while running:
    running = scene.render()
    # Wind impulse at step 60
    if step == 60:
        for r in range(rows):
            for c in range(cols):
                if sim.particles[ids[r,c]].inv_mass > 0:
                    sim.particles[ids[r,c]].vel += np.array([2.0, 0.0, 3.0])
    sim.step(dt)
    view = gWindow._myCamera

# Update particle spheres
for r,c,pid,tr,sh in part_data:
    pos = sim.particles[pid].pos

```

```

        mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

    identity = util.identity()
   .mvp_e = proj @ view @ identity
    for tr_e, sh_e, ev, r, c in h_edges:
        ev[0,:3] = sim.particles[ids[r,c]].pos
        ev[1,:3] = sim.particles[ids[r,c+1]].pos
        sh_e.setUniformVariable(key='modelViewProj', value=mvp_e, mat4=True)
    for tr_e, sh_e, ev, r, c in v_edges:
        ev[0,:3] = sim.particles[ids[r,c]].pos
        ev[1,:3] = sim.particles[ids[r+1,c]].pos
        sh_e.setUniformVariable(key='modelViewProj', value=mvp_e, mat4=True)

    KE = sim.kinetic_energy()
    displayGUI_text(
        f"XR-PBD 3D Cloth ({rows}x{cols} grid)\n"
        f"Particles: {rows*cols} | Constraints: {len(sim.constraints)}\n"
        f"KE={KE:.3f} J | Step {step}\n"
        f"Wind impulse at t=60 frames | SOR w=1.8\n"
        f"ESC to exit"
    )
    scene.world.traverse_visit(render_sys, scene.world.root)
    scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
    scene.world.traverse_visit(cam_sys, scene.world.root)
    scene.render_post()
    step += 1

    scene.shutdown()
    print(f" 3D Cloth: {rows*cols} particles, {len(sim.constraints)}
↪constraints, {step} steps")

print("run_cloth_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_cloth_3d(rows=10, cols=10)

```

## 1.7 Section 4: 3D Rigid Body — Shape Matching in Elements

**Shape matching** extracts a rotation matrix  $R$  via SVD polar decomposition of the deformation gradient, then pulls all particles toward their rotated rest positions.

In this demo, a  $3 \times 3 \times 3$  cube of particles (27 nodes) is launched with angular velocity and collides with a floor plane.

```
[ ]: class ShapeMatchingConstraint3D:
    """
    3D shape matching constraint (Müller 2005) for rigid-body-like behaviour.
    Pulls particles toward SVD-extracted rotated rest positions.
    """
    def __init__(self, particle_ids, particles, stiffness=0.95):
        self.ids = particle_ids
        self.particles = particles
        self.stiffness = stiffness
        poses = np.array([particles[i].pos for i in particle_ids])
        self.rest_offsets = poses - poses.mean(0)

    def reset_lambda(self): pass

    def solve(self, h, sor=1.0):
        poses = np.array([self.particles[i].pos for i in self.ids])
        com = poses.mean(0)
        offsets = poses - com
        Apq = offsets.T @ self.rest_offsets
        try:
            U, _, Vt = np.linalg.svd(Apq)
        except np.linalg.LinAlgError:
            return
        R = U @ Vt
        if np.linalg.det(R) < 0:
            U[:, -1] *= -1; R = U @ Vt
        targets = (R @ self.rest_offsets.T).T + com
        for pid, tgt in zip(self.ids, targets):
            self.particles[pid].pos += (tgt - self.particles[pid].pos) * self.
↪stiffness * sor

    def run_rigid_3d(dt=0.016):
        """3D rigid-body shape matching in Elements scenegraph."""
        sim = XRPBDSimulator(substeps=6, iterations=6, sor_factor=1.5,
                               gravity=[0., -9.81, 0.])
        # 3x3x3 particle cube
        n = 3; spacing = 0.4
        pids = []
        for ix in range(n):
            for iy in range(n):
                for iz in range(n):
                    x = (ix - n//2) * spacing
                    y = (iy - n//2) * spacing + 2.0
                    z = (iz - n//2) * spacing
                    pid = sim.add_particle([x,y,z], 0.8)
                    pids.append(pid)
        # Angular velocity (spin)
```

```

com = np.mean([sim.particles[p].pos for p in pids], axis=0)
for pid in pids:
    r = sim.particles[pid].pos - com
    omega = np.array([2.0, 3.0, 1.0]) # angular velocity vector
    sim.particles[pid].vel = np.cross(omega, r)
    sim.add_constraint(ShapeMatchingConstraint3D(pids, sim.particles,
↪stiffness=0.95))

# Build Scene
scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Rigid Body")
trans_sys = scene.world.
↪createSystem(TransformSystem("ts", "TransformSystem", "001"))
cam_sys = scene.world.
↪createSystem(CameraSystem("cs", "CameraSystem", "200"))
render_sys = scene.world.createSystem(RenderGLShaderSystem())
init_sys = scene.world.createSystem(InitGLShaderSystem())

sV, sC, sI = make_sphere_mesh(r=0.12, lats=6, lons=8)
part_data = []
for i, pid in enumerate(pids):
    pos = sim.particles[pid].pos
    r,g,b = (i%3)*0.3+0.1, ((i//3)%3)*0.3+0.1, ((i//9)%3)*0.3+0.1
    col = np.tile([r,g,b,1.0],(len(sV),1)).astype(np.float32)
    ent, tr, sh = add_particle_entity(scene, root, f"rb_{i}", pos, sV, sC,
↪sI, color=col)
    part_data.append((pid, tr, sh))

scene.init(imgui=True, windowWidth=1024, windowHeight=768,
           windowTitle="XR-PBD 3D Rigid Body - Shape Matching",
           customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager = scene.world.eventManager; gWindow = scene.renderWindow
rgl = RenderGLStateSystem()
for ev_name in ['OnUpdateCamera', 'OnUpdateWireframe']:
    eManager._subscribers[ev_name] = gWindow
    eManager._actuators[ev_name] = rgl
gWindow._myCamera = util.lookat(util.vec(4,3,4), util.vec(0,1,0), util.
↪vec(0,1,0))

step = 0; running = True
while running:
    running = scene.render()
    sim.step(dt)
    # Floor
    for pid in pids:
        pt = sim.particles[pid]

```

```

        if pt.pos[1] < -2.0:
            pt.pos[1] = -2.0; pt.vel[1] = abs(pt.vel[1]) * 0.4
view = gWindow._myCamera
KE = sim.kinetic_energy()

for pid, tr, sh in part_data:
    pos = sim.particles[pid].pos
    mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
    tr.trs = mdl
    sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↳mat4=True)

displayGUI_text(
    f"XR-PBD 3D Rigid Body - Shape Matching (SVD)\n"
    f"Particles: {n**3} (3x3x3 grid) | Stiffness=0.95\n"
    f"KE={KE:.3f} J | Step {step}\n"
    f"ESC to exit"
)
scene.world.traverse_visit(render_sys, scene.world.root)
scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
scene.world.traverse_visit(cam_sys, scene.world.root)
scene.render_post()
step += 1

scene.shutdown()
print(f" 3D Rigid body: {n**3} particles, {step} steps, final KE={KE:.4f}")

print("run_rigid_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_rigid_3d()

```

## 1.8 Section 5: 3D Tearing Simulation in Elements

**Breakable distance constraints:** each constraint estimates  $F \approx |\Delta\lambda|/\Delta t^2$ . When  $F > F_{threshold}$ , the constraint is deactivated.

The broken edges turn **red** in the Elements scenegraph to highlight the tear.

```

[ ]: def run_tearing_3d(n=20, break_force=4.0, dt=0.016):
    """3D rope tearing in Elements scenegraph."""
    sim = XRPBDSimulator(substeps=6, iterations=8, sor_factor=1.8,
                          gravity=[0., -4.0, 0.]) # softer gravity to see tear
    for i in range(n):
        sim.add_particle([0.0, 3.5 - i*0.18, 0.0], mass=1.0)
    sim.particles[0].inv_mass = 0 # pin top
    constraints = []
    for i in range(n-1):

```



```

        c = DistanceConstraint(i, i+1, sim.particles, alpha=1e-5,
                               breakable=True, break_force=break_force)
        sim.add_constraint(c); constraints.append(c)
# Heavy mass at bottom
        sim.particles[-1].mass = 12.0; sim.particles[-1].inv_mass = 1/12.0

# Build Scene
        scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Tearing")
        trans_sys = scene.world.
↪createSystem(TransformSystem("ts","TransformSystem","001"))
        cam_sys = scene.world.
↪createSystem(CameraSystem("cs","CameraSystem","200"))
        render_sys = scene.world.createSystem(RenderGLShaderSystem())
        init_sys = scene.world.createSystem(InitGLShaderSystem())

        sV, sC, sI = make_sphere_mesh(r=0.055, lats=5, lons=6)

        part_data = []
        for i in range(n):
            pos = sim.particles[i].pos
            pin = (sim.particles[i].inv_mass == 0)
            col = np.tile([1.0,0.5,0.0,1.0] if pin else [0.2,0.8,0.8,1.0],
                           (len(sV),1)).astype(np.float32)
            ent, tr, sh = add_particle_entity(scene, root, f"tr_{i}", pos, sV, sC,
↪sI, color=col)
            part_data.append((tr, sh))

        edge_data = []
        for i, c in enumerate(constraints):
            p0 = sim.particles[i].pos
            p1 = sim.particles[i+1].pos
            _, tr_e, sh_e, ev = add_edge_entity(scene, root, f"e_{i}", p0, p1)
            edge_data.append((tr_e, sh_e, ev, c))

        scene.init(imgui=True, windowWidth=1024, windowHeight=768,
                   windowTitle="XR-PBD 3D Tearing - Breakable Constraints",
                   customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
        scene.world.traverse_visit(init_sys, scene.world.root)

        eManager = scene.world.eventManager; gWindow = scene.renderWindow
        rgl = RenderGLStateSystem()
        for ev_name in ['OnUpdateCamera','OnUpdateWireframe']:
            eManager._subscribers[ev_name] = gWindow
            eManager._actuators[ev_name] = rgl
        gWindow._myCamera = util.lookat(util.vec(2,2,3), util.vec(0,1.5,0), util.
↪vec(0,1,0))

```

```

step = 0; running = True
while running:
    running = scene.render()
    sim.step(dt)
    view = gWindow._myCamera
    n_broken = sum(1 for c in constraints if not c.active)
    mvp_e = proj @ view @ util.identity()

    for i, (tr, sh) in enumerate(part_data):
        pos = sim.particles[i].pos
        mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

        for tr_e, sh_e, ev, c in edge_data:
            ev[0,:3] = sim.particles[c.i1].pos
            ev[1,:3] = sim.particles[c.i2].pos
            # Broken edges turn red
            if not c.active:
                sh_e.setUniformVariable(key='modelViewProj', value=mvp_e * 0,
↪mat4=True) # hide broken
            else:
                sh_e.setUniformVariable(key='modelViewProj', value=mvp_e,
↪mat4=True)

        displayGUI_text(
            f"XR-PBD 3D Tearing\n"
            f"Break threshold: {break_force} N | Broken: {n_broken}/{n-1}\n"
            f"Step {step} | Heavy bottom mass = 12kg\n"
            f"ESC to exit"
        )
    scene.world.traverse_visit(render_sys, scene.world.root)
    scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
    scene.world.traverse_visit(cam_sys, scene.world.root)
    scene.render_post()
    step += 1

scene.shutdown()
n_broken = sum(1 for c in constraints if not c.active)
print(f" 3D Tearing: broke {n_broken}/{n-1} constraints ({step} steps)")

print("run_tearing_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_tearing_3d(n=20, break_force=4.0)

```

## 1.9 Section 6: 3D Needle Piercing — Insertion Constraint in Elements

The **novel XR-PBD contribution** (SIGGRAPH Asia 2025, §4.4): A 5-particle constraint couples a moving needle segment  $(n_0, n_1)$  to a triangular membrane face  $(t_0, t_1, t_2)$ .

$$q = w_0x_{t_0} + w_1x_{t_1} + w_2x_{t_2}, \quad r = (1 - v)n_0 + vn_1, \quad C = |q - r|$$

In the Elements scene: - **Cyan spheres** = tissue membrane vertices (3 pinned) - **Orange cylinder** (GL\_LINES) = needle - Green line = constraint connection (q→r) drawn realtime

```
[ ]: def run_piercing_3d(dt=0.016):
    """3D needle piercing simulation in Elements scenegraph."""
    sim = XRPBDSimulator(substeps=8, iterations=10, sor_factor=1.8,
                          gravity=[0., -9.81, 0.])
    # Triangular membrane (3 pinned vertices + 1 dynamic center)
    mem_pts = [[-0.9, 0., 0.], [0.9, 0., 0.], [0., 0., 1.2], [0., 0.15, 0.4]]
    mem_ids = [sim.add_particle(pt, 0.8) for pt in mem_pts]
    for i in range(3): sim.particles[mem_ids[i]].inv_mass = 0 # pin triangle
    for a, b in [(0, 3), (1, 3), (2, 3), (0, 1), (1, 2), (2, 0)]:
        sim.add_constraint(DistanceConstraint(mem_ids[a], mem_ids[b], sim.
        ↪particles, alpha=1e-5))

    # Needle: 2 particles, moving downward
    n0 = sim.add_particle([0.0, 3.0, 0.4], 2.0)
    n1 = sim.add_particle([0.0, 2.0, 0.4], 2.0)
    sim.particles[n0].vel = np.array([0., -1.8, 0.])
    sim.particles[n1].vel = np.array([0., -1.8, 0.])
    sim.add_constraint(DistanceConstraint(n0, n1, sim.particles, alpha=1e-7))
    # Insertion constraint: needle into membrane center
    ic = InsertionConstraint(mem_ids[0], mem_ids[1], mem_ids[2],
                            n0, n1, sim.particles,
                            bw0=0.33, bw1=0.34, bw2=0.33, v=0.75,
                            alpha=5e-4)
    sim.add_constraint(ic)

    # Build Scene
    scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Piercing")
    trans_sys = scene.world.
    ↪createSystem(TransformSystem("ts", "TransformSystem", "001"))
    cam_sys = scene.world.
    ↪createSystem(CameraSystem("cs", "CameraSystem", "200"))
    render_sys = scene.world.createSystem(RenderGLShaderSystem())
    init_sys = scene.world.createSystem(InitGLShaderSystem())

    sV, sC, sI = make_sphere_mesh(r=0.07, lats=6, lons=8)

    # Membrane sphere entities
```

```

mem_data = []
for i, mid in enumerate(mem_ids):
    pos = sim.particles[mid].pos
    col = np.tile([0.0,0.9,0.9,1.0] if i<3 else [0.0,1.0,0.4,1.0],
                  (len(sV),1)).astype(np.float32)
    ent, tr, sh = add_particle_entity(scene, root, f"mem_{i}", pos, sV, sC,
    ↪sI, color=col)
    mem_data.append((mid, tr, sh))

# Membrane edges
mem_edges = []
for a,b in [(0,3),(1,3),(2,3),(0,1),(1,2),(2,0)]:
    p0 = sim.particles[mem_ids[a]].pos
    p1 = sim.particles[mem_ids[b]].pos
    _, tr_e, sh_e, ev = add_edge_entity(scene, root, f"me_{a}-{b}", p0, p1)
    mem_edges.append((tr_e, sh_e, ev, a, b))

# Needle entities (n0 sphere orange, n1 sphere orange, line between)
sVN, _, sIN = make_sphere_mesh(r=0.08, lats=5, lons=7)
col_needle = np.tile([1.0,0.4,0.0,1.0],(len(sVN),1)).astype(np.float32)
_, tr_n0, sh_n0 = add_particle_entity(scene, root, "ndl0", sim.
    ↪particles[n0].pos, sVN, col_needle.copy(), sIN, color=col_needle)
_, tr_n1, sh_n1 = add_particle_entity(scene, root, "ndl1", sim.
    ↪particles[n1].pos, sVN, col_needle.copy(), sIN, color=col_needle)
_, tr_nl, sh_nl, ev_nl = add_edge_entity(scene, root, "ndl_e",
    sim.particles[n0].pos, sim.
    ↪particles[n1].pos)

# Constraint visualization line
_, tr_cv, sh_cv, ev_cv = add_edge_entity(scene, root, "cons_vis",
    sim.particles[n0].pos, sim.
    ↪particles[n0].pos)

scene.init(imgui=True, windowWidth=1024, windowHeight=768,
           windowTitle="XR-PBD 3D Insertion Constraint - Needle Piercing",
           customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager = scene.world.eventManager; gWindow = scene.renderWindow
rgl = RenderGLStateSystem()
for ev_name in ['OnUpdateCamera', 'OnUpdateWireframe']:
    eManager._subscribers[ev_name] = gWindow
    eManager._actuators[ev_name] = rgl
gWindow._myCamera = util.lookat(util.vec(3,2,3), util.vec(0,0.5,0), util.
    ↪vec(0,1,0))

step = 0; running = True

```

```

while running:
    running = scene.render()
    sim.step(dt)
    view = gWindow._myCamera
    mvp_I = proj @ view @ util.identity()

    # Compute q,r for visualization
    tp = [sim.particles[m].pos for m in mem_ids[:3]]
    q = 0.33*np.array(tp[0]) + 0.34*np.array(tp[1]) + 0.33*np.array(tp[2])
    r = 0.25*sim.particles[n0].pos + 0.75*sim.particles[n1].pos
    gap = np.linalg.norm(q-r)

    for mid, tr, sh in mem_data:
        pos = sim.particles[mid].pos
        mdl = util.translate(float(pos[0]),float(pos[1]),float(pos[2]))
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

    for tr_e, sh_e, ev, a, b in mem_edges:
        ev[0,:3] = sim.particles[mem_ids[a]].pos
        ev[1,:3] = sim.particles[mem_ids[b]].pos
        sh_e.setUniformVariable(key='modelViewProj', value=mvp_I, mat4=True)

    for (pid, tr, sh) in [(n0, tr_n0, sh_n0), (n1, tr_n1, sh_n1)]:
        pos = sim.particles[pid].pos
        mdl = util.translate(float(pos[0]),float(pos[1]),float(pos[2]))
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

    ev_n1[0,:3] = sim.particles[n0].pos
    ev_n1[1,:3] = sim.particles[n1].pos
    sh_n1.setUniformVariable(key='modelViewProj', value=mvp_I, mat4=True)
    ev_cv[0,:3] = q; ev_cv[1,:3] = r
    sh_cv.setUniformVariable(key='modelViewProj', value=mvp_I, mat4=True)

    displayGUI_text(
        f"XR-PBD 3D Insertion Constraint (Piercing)\n"
        f"|q - r| = {gap:.5f} (constraint residual)\n"
        f"Cyan = membrane | Orange = needle\n"
        f"Green line = insertion constraint q-r\n"
        f"Step {step} | ESC to exit"
    )

    scene.world.traverse_visit(render_sys, scene.world.root)
    scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
    scene.world.traverse_visit(cam_sys, scene.world.root)

```

```

        scene.render_post()
        step += 1

    scene.shutdown()
    print(f" 3D Piercing: final |q-r|={gap:.5f} ({step} steps)")

print("run_piercing_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_piercing_3d()

```

## 1.10 Quick Launcher

Run any simulation interactively:

| Demo       | Function                       | Description                   |
|------------|--------------------------------|-------------------------------|
| Rope       | <code>run_rope_3d()</code>     | 16-particle rope with swing   |
| Soft Body  | <code>run_softbody_3d()</code> | Tetrahedral volume constraint |
| Cloth      | <code>run_cloth_3d()</code>    | 10×10 grid with wind          |
| Rigid Body | <code>run_rigid_3d()</code>    | 27-particle spinning box      |
| Tearing    | <code>run_tearing_3d()</code>  | Breakable rope under load     |
| Piercing   | <code>run_piercing_3d()</code> | Novel insertion constraint    |

```

[ ]: # Interactive launcher
DEMOS = {
    '1': ('Rope', run_rope_3d),
    '2': ('Soft Body', run_softbody_3d),
    '3': ('Cloth', run_cloth_3d),
    '4': ('Rigid', run_rigid_3d),
    '5': ('Tearing', run_tearing_3d),
    '6': ('Piercing', run_piercing_3d),
}
print("Available XR-PBD 3D demos in Elements Scenegraph:")
for k,(name,fn) in DEMOS.items():
    print(f" {k}: {name}")
print()
print("Usage: DEMOS['1'][1]() or run_rope_3d()")

```